TP3 : Prise en main de Spark SQL

SparkSession, DataFrames, requêtes SQL et formats de fichiers

Partie 0 – Préparation des données

Dans un notebook Jupyter, exécutez la cellule suivante. Elle importe les modules nécessaires et crée les fichiers de données utilisés pendant le TP (les fichiers people.json et people.txt du cours, ainsi qu'un fichier employes.csv).

Cellule à exécuter :

```
import os
from pyspark.sql import SparkSession, Row
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

os.makedirs("data_tp1", exist_ok=True)

# Fichiers du cours (people)
with open("data_tp1/people.json", "w", encoding="utf-8") as f:
    f.write('{"name":"Michael"}\n{"name":"Andy", "age":30}\n')
    f.write('{"name":"Justin", "age":19}\n')
with open("data_tp1/people.txt", "w", encoding="utf-8") as f:
    f.write("Michael, 29\nAndy, 30\nJustin, 19\n")

# Données des employés
employes = [
    (1, "Sara",    "Informatique", "Casablanca", 14000, 28),
    (2, "Youssef", "Informatique", "Rabat",      16500, 35),
    (3, "Imane",   "Finance",      "Casablanca", 12000, 41),
    (4, "Omar",    "Finance",      "Tanger",      9500, 26),
    (5, "Salma",   "RH",           "Rabat",       8800, 30),
    (6, "Hamza",   "Informatique", "Marrakech",  11000, 24),
    (7, "Khadija", "RH",           "Casablanca", 10200, 38),
    (8, "Anas",    "Finance",      "Rabat",      13500, 45),
]
colonnes = ["id", "nom", "departement", "ville", "salaire", "age"]

with open("data_tp1/employes.csv", "w", encoding="utf-8") as f:
    f.write(",".join(colonnes) + "\n")
    for e in employes:
        f.write(",".join(str(x) for x in e) + "\n")
print("Fichiers créés :", os.listdir("data_tp1"))
```

Partie 1 – SparkSession

La SparkSession est le point d'entrée de l'API DataFrame/Dataset depuis Spark 2.0. Elle se crée via l'objet builder.

Question 1.1

Créez une SparkSession nommée « TP1 - Spark SQL », en mode local utilisant tous les cœurs, avec la configuration spark.sql.shuffle.partitions = 4. Récupérez aussi le SparkContext dans une variable sc.

Indice : *SparkSession.builder.appName(...).master("local[*]").config(...).getOrCreate()*

Question 1.2

Affichez la version de Spark, le nom de l'application, le master, et la valeur de spark.sql.shuffle.partitions.

Indice : *spark.version, sc.appName, sc.master, spark.conf.get(...)*

Question 1.3

Spark 4 active par défaut le mode ANSI SQL. Affichez la valeur de spark.sql.ansi.enabled. Que change ce mode pour une conversion de type invalide (par exemple CAST('abc' AS INT)) ?

Indice : *Testez : spark.sql("SELECT CAST('abc' AS INT)").show() dans un bloc try/except, puis spark.sql("SELECT try_cast('abc' AS INT)").show().*

Partie 2 – DataFrame à partir d'une séquence

Un DataFrame est une collection distribuée et immuable de données organisées en colonnes nommées, comme une table relationnelle. La liste employes a été définie dans la partie 0.

Question 2.1

Créez un DataFrame df_emp à partir de la liste employes et de la liste colonnes. Affichez son contenu et son schéma.

Indice : *spark.createDataFrame(donnees, colonnes), puis show() et printSchema().*

Question 2.2

Affichez les noms de colonnes, les types (dtypes), le nombre de lignes, puis les statistiques descriptives de la colonne salaire.

Indice : *df.columns, df.dtypes, df.count(), df.describe("salaire").show()*

Question 2.3

Recréez le même DataFrame (df_emp2) avec un schéma explicite de type StructType : id (entier, non nul), nom, departement, ville (chaînes), salaire et age (entiers). Comparez le schéma obtenu avec celui de la question 2.1.

Indice : *StructType([StructField("id", IntegerType(), False), ...])*

Partie 3 – DataFrame à partir de sources de données

L'objet spark.read (classe DataFrameReader) charge des données depuis un système de stockage externe. Les méthodes json(), csv(), parquet()… sont des cas particuliers de la méthode générique load().

Question 3.1

Chargez data_tp1/people.json dans un DataFrame df_people_json. Affichez-le et affichez son schéma. Que remarquez-vous pour Michael ? Quel est le type de la colonne age ?

Indice : *spark.read.json(chemin)*

Question 3.2

Chargez data_tp1/employes.csv de deux façons : (a) avec l'option header seulement, (b) avec header et inferSchema. Comparez les deux schémas.

Indice : *spark.read.option("header", True).option("inferSchema", True).csv(chemin)*

Question 3.3

Rechargez le CSV avec la méthode générique : spark.read.format(...).option(...).load(...). Vérifiez que le résultat est identique à df_csv_b.

Indice : *df1.exceptAll(df2).count() == 0 permet de comparer le contenu de deux DataFrames.*

Question 3.4

Pourquoi l'option inferSchema est-elle déconseillée sur de gros volumes ? Quelle alternative recommandez-vous ?

Réponse : ……

Partie 4 – Opérations sur DataFrame

L'API DataFrame propose des opérateurs relationnels (select, where, join, groupBy) de type transformation (évaluation paresseuse) et des actions (show, count, collect) qui déclenchent le calcul. Travaillez sur df_emp.

Question 4.1

Sélectionnez les colonnes nom et salaire. Ajoutez ensuite une colonne salaire_annuel égale à salaire * 12.

Indice : *select("nom", "salaire") puis select(..., (F.col("salaire") * 12).alias("salaire_annuel"))*

Question 4.2

Affichez les employés du département Informatique dont le salaire dépasse 12 000.

Indice : *filter / where avec l'opérateur & (parenthèses obligatoires autour de chaque condition).*

Question 4.3

Ajoutez une colonne profil avec withColumn : « Junior » si age < 30, « Confirmé » si age < 40, « Senior » sinon.

Indice : *F.when(cond1, v1).when(cond2, v2).otherwise(v3)*

Question 4.4

Pour chaque département, calculez le nombre d'employés, le salaire moyen (arrondi à 0 décimale) et le salaire maximum.

Indice : *groupBy("departement").agg(F.count("*"), F.avg(...), F.max(...)) avec alias.*

Question 4.5

Affichez les trois employés les mieux payés, triés par salaire décroissant.

Indice : *orderBy(F.desc("salaire")).limit(3)*

Partie 5 – Requêtes SQL

Spark SQL permet d'exécuter des requêtes SQL sur un DataFrame enregistré comme vue temporaire. spark.sql() retourne un DataFrame.

Question 5.1

Enregistrez df_emp comme vue temporaire employes. Écrivez une requête SQL qui retourne, pour chaque département, le salaire moyen, uniquement pour les départements dont la moyenne dépasse 10 000, triés par moyenne décroissante.

Indice : *df.createOrReplaceTempView("employes") puis spark.sql("SELECT ... GROUP BY ... HAVING ... ORDER BY ...")*

Question 5.2

Une vue temporaire n'est visible que dans la session qui l'a créée. Créez une vue globale emp_global, puis interrogez-la depuis une nouvelle session (spark.newSession()). Vérifiez que la vue locale « employes » n'est pas visible depuis cette nouvelle session.

Indice : *createGlobalTempView, table qualifiée par global_temp.*

Question 5.3

Exécutez une requête SQL directement sur le fichier people.json, sans créer de DataFrame ni de vue : SELECT * FROM json.`chemin`. Filtrez les personnes de plus de 20 ans.

Indice : *Le chemin est entouré d'accents graves (backticks).*

Question 5.4

Écrivez la requête de la question 4.4 (agrégation par département) en SQL, puis comparez les plans physiques obtenus avec explain() pour la version DataFrame et la version SQL. Que constatez-vous et pourquoi ?

Indice : *df.explain() et spark.sql(...).explain()*

Partie 6 – Conversions entre RDD et DataFrame

Un RDD peut être transformé en DataFrame de deux façons : par inférence du schéma (à partir d'objets Row) ou en spécifiant explicitement un schéma. Le fichier people.txt contient des lignes « Nom, age ».

Question 6.1

Lisez people.txt avec sc.textFile, transformez chaque ligne en objet Row(name=..., age=...) (age converti en entier), puis créez un DataFrame df_people par inférence.

Indice : *l.split(","), int(p[1].strip()), spark.createDataFrame(rdd_rows)*

Question 6.2

Créez le même DataFrame en spécifiant le schéma : name (string, non nul) et age (int). Les éléments du RDD sont cette fois des tuples.

Indice : *spark.createDataFrame(rdd_tuples, schema)*

Question 6.3

Convertissez df_people en RDD via l'attribut rdd. Affichez deux éléments avec take(2). Quel est le type des éléments ? Comment accéder au champ name d'un élément ?

……